Hardware-Aware CNN Optimization Toolkit: Automated Pruning, Convolution Transformation, with Heterogeneous Layer Scheduling on Intel Lunar Lake AI PC

Ritik Agarwal
MTech DSBA
Indian Institute of Science, Bengaluru
Email: ritikagarwal@iisc.ac.in

***Abstract*—This project proposes to efficiently use heterogeneous systems like Intel Lunar Lake AI PC by providing an automated hardware-aware optimization toolkit for deploying convolutional neural networks across CPU, GPU, and NPU. The toolkit takes a pretrained CNN as input and creates different variants of the CNN by applying structured filter pruning, optional dense to depthwise-separable-convolution conversion, and INT8 quantization. Each variant is thoroughly profiled across various parameters that will be discussed, before deploying each layer on the most suitable device available. The goal is to maximize performance and efficiency while maintaining a target accuracy. A Python implementation built on PyTorch, OpenVINO, and NNCF will be evaluated on multiple CNN architectures and optimization configurations, reporting speedup, energy, and accuracy trade-offs under different deployment strategies.**

***Index Terms*—Convolutional neural networks, model compression, pruning, quantization, heterogeneous computing, Intel Lunar Lake, OpenVINO, NNCF.**

I. Introduction

Deep convolutional neural networks (CNNs) are widely used in computer vision and classification workloads, but their computational and memory demands pose a challenge for energy and latency constrained systems. Early CNNs like LeNet-5 [1], which was built using just 5 layers and only 2.6K weights, do not require that much compute. However, the more recent ones, such as GoogleNet [2] with 6M weights or ResNet-50 [3], which exceeds 23M parameters, are much more compute-hungry. To maintain the cost-effectiveness of such neural networks, they need to be efficiently deployed and managed across both hardware and software levels.

Modern client and edge platforms, such as Intel Lunar Lake, provide heterogeneous compute resources combining general-purpose CPU cores, integrated GPUs, and dedicated NPUs for AI acceleration. CPUs are designed for general-purpose workloads, whereas GPUs offer a high degree of parallelism but at the cost of higher power consumption. NPUs can execute a high number of operations simultaneously [4], while consuming much lower power than GPUs.

However, mapping an unoptimized CNN directly onto this heterogeneous hardware often underutilizes the compute available. Deploying a CNN completely on a GPU incurs high power consumption, while a CPU deployment drastically reduces performance. Most neural networks can also benefit from certain techniques, using which different layers can leverage the heterogeneous compute on the device to provide the most optimal throughput and power efficiency.

This project targets the problem of automatically transforming and scheduling CNNs to exploit Lunar Lake's CPU–GPU–NPU ensemble. Different CNNs will be studied and profiled based on the number of weights, MAC operations, and power consumption to understand which techniques can provide the most benefit for different hardware options. The focus is on building a practical end-to-end Python toolkit that can be applied to arbitrary CNN models, rather than on proposing a single new compression algorithm.

In this Phase 1 report, the emphasis is on defining the problem, surveying related work in model compression and heterogeneous deployment, and outlining the technical plan and quantitative goals for the full project.

II. Related Work

Research on CNN model compression has explored pruning, quantization [5], [6], and architecture transformations to reduce computation and memory footprint while preserving accuracy. Deep work has been done on structured pruning [7] to solve these challenges further.

Quantization techniques, including post-training quantization [8] and quantization-aware training [9], enable deployment of CNNs in reduced precision (e.g., INT8) for improved throughput and energy efficiency on accelerators and CPUs, while maintaining target accuracy. However, not all precisions are supported across all devices. In this project, we split the layers onto compatible devices while considering this factor.

Depthwise separable convolutions [10], popularized by families such as MobileNet and Xception, restructure convolution layers to lower MAC counts and parameter counts. Transforming a dense layer to depthwise separable theoretically reduces the number of parameters, but it can also end up under-utilizing the memory bandwidth [11]. We analyze which

layers can benefit in terms of performance from dense-to-DSC conversion, while hitting accuracy targets.

Hardware-aware optimization frameworks and compilers have emerged to automatically tailor neural networks to target platforms. We use OpenVINO on Intel AIPC, which integrates quantization, pruning, and heterogeneous deployment [12] of models across devices. However, the layer splitting is based on general heuristics, such as compatibility and user preference. We extend this capability by conducting layer-level profiling across multiple permutations and combinations to provide the most optimized and efficient deployment.

There is growing work on heterogeneous device placement and scheduling of DNN operators across multiple devices [13]. Profiling-driven schedulers using dynamic programming-based algorithms have been developed [14] to assign operators to devices to minimize latency or energy consumption subject to communication constraints. These algorithms enable us to decide layer affinity by using our calculations as input.

III. Brief Plan of Action

The project will be developed in multiple phases centered around a profiling-driven optimization pipeline.

First, a profiling backend will be implemented that ingests an OpenVINO Intermediate Representation (IR) model, iterates through the layers and operation types, and computes analytical metrics such as MAC counts, estimated memory traffic, energy consumption, and arithmetic intensity per layer. This backend will also execute the model on Lunar Lake CPU, iGPU, and NPU devices to obtain per-device latency measurements and compatibility information of the layers.

Second, three optimization transforms will be integrated into a Python pipeline: (i) structured filter pruning using NNCF in PyTorch with configurable pruning rates and fine-tuning the pruned model; (ii) optional dense-to-depthwise-separable convolution conversion for selected layers based on MAC reduction and effect on accuracy; and (iii) post-training INT8 quantization using OpenVINO tooling applied after pruning and convolution transformation to reduce the final model size while meeting target accuracy.

Third, a layer-to-device scheduler will be designed that takes per-layer latency and inter-device transfer cost as input and computes a near-optimal assignment of CNN layers to CPU, GPU, or NPU. We will implement a greedy heuristic approach initially, followed by a DP-based scheduler to allocate sets of continuous layers to corresponding devices.

Finally, these components will be exposed through a command-line interface (CLI) that automates CNN loading, conversion to OpenVINO IR, profiling, scheduling, heterogeneous deployment, and benchmarking on Lunar Lake. Experimental evaluation will cover multiple CNN architectures and combinations of pruning, DSC conversion, quantization, and device placement strategies.

IV. Final Goal

The final objective of the project is to demonstrate that the proposed toolkit can deliver significant performance and efficiency gains for CNN inference on Intel Lunar Lake, compared to vanilla deployments, while maintaining acceptable accuracy.

More specifically, the goal is to achieve, for at least two representative CNN models, a heterogeneous deployment configuration (combining pruning, DSC, and INT8 quantization) that attains:

- At least $1.5 - 3\times$ reduction in end-to-end latency compared to an unoptimized FP16 all-CPU baseline.
- At least $2\times$ improvement in energy efficiency (inferences per joule) relative to an all-GPU baseline.
- A maximum top-1 accuracy degradation of no more than $1.5 - 2.5\%$ on the original evaluation dataset.

The project also aims to produce speedup matrices, preference of devices for different kinds of layers, and accuracy–performance visualizations that quantify how pruning, DSC conversion, quantization, and heterogeneous layer scheduling come together to yield non-linear performance gains.

References

[1] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[2] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. E. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," *CoRR*, vol. abs/1409.4842, 2014. [Online]. Available: http://arxiv.org/abs/1409.4842

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," 2015. [Online]. Available: https://arxiv.org/abs/1512.03385

[4] T. Tan and G. Cao, "Efficient execution of deep neural networks on mobile devices with npu," in *Proceedings of the 20th International Conference on Information Processing in Sensor Networks (Co-Located with CPS-IoT Week 2021)*, ser. IPSN '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 283–298. [Online]. Available: https://doi.org/10.1145/3412382.3458272

[5] A. Kuzmin, M. Nagel, M. van Baalen, A. Behboodi, and T. Blankevoort, "Pruning vs quantization: Which is better?" in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 62 414–62 427. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/c48bc80aa5d3cbbdd712d1cc107b8319-Paper-Conference.pdf

[6] Q. Qi, Y. Lu, J. Li, J. Wang, H. Sun, and J. Liao, "Learning low resource consumption cnn through pruning and quantization," *IEEE Transactions on Emerging Topics in Computing*, vol. 10, no. 2, pp. 886–903, 2022.

[7] Y. He and L. Xiao, "Structured pruning for deep convolutional neural networks: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 5, pp. 2900–2919, 2024.

[8] P. Kluska and M. Zieba, "Post-training quantization methods for deep learning models," in *Intelligent Information and Database Systems*, N. T. Nguyen, K. Jearanaitanakij, A. Selamat, B. Trawiński, and S. Chittayasothorn, Eds. Cham: Springer International Publishing, 2020, pp. 467–479.

[9] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, "A white paper on neural network quantization," 2021. [Online]. Available: https://arxiv.org/abs/2106.08295

[10] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," 2017. [Online]. Available: https://arxiv.org/abs/1610.02357

[11] R. Shang, J. He, J. Wang, K. Xu, L. Jiao, and R. Stolkin, "Dense connection and depthwise separable convolution based cnn for polarimetric sar image classification," *Knowledge-Based Systems*, vol. 194, p. 105542, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0950705120300411

[12] F. Z. Guerrouj, M. ABOUZAHIR, M. RAMZI, and E. M. ABDALI, "Analysis of the acceleration of deep learning inference models on a heterogeneous architecture based on openvino," in *2021 4th International Symposium on Advanced Electrical and Communication Technologies (ISAECT)*, 2021, pp. 01–05.

[13] Z. Xu, D. Yang, C. Yin, J. Tang, Y. Wang, and G. Xue, "A co-scheduling framework for dnn models on mobile and edge devices with heterogeneous hardware," *IEEE Transactions on Mobile Computing*, vol. 22, no. 3, pp. 1275–1288, 2023.

[14] J. Tarnawski, A. Phanishayee, N. R. Devanur, D. Mahajan, and F. N. Paravecino, "Efficient algorithms for device placement of dnn graph operators," 2020. [Online]. Available: https://arxiv.org/abs/2006.16423